

UNIT II PROCESS MANAGEMENT

Processes - Process Concept - Process Scheduling - Operations on Processes - Inter-process Communication; Threads - Multithread Models – Threading issues- Thread libraries; CPU Scheduling – Basic Concepts- Scheduling criteria - Scheduling algorithms – multiprocessor scheduling

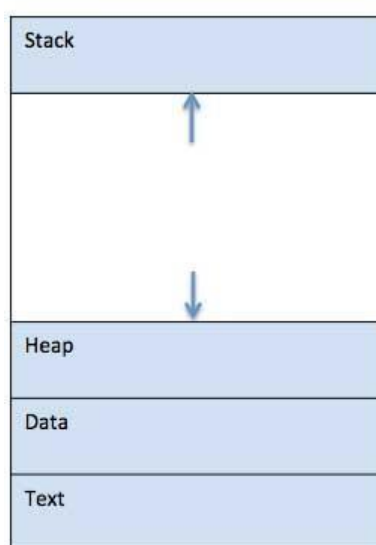
PROCESSES

PROCESS CONCEPT

A process is basically a program in execution. The execution of a process must progress in a sequential fashion.

A process is defined as an entity which represents the basic unit of work to be implemented in the system.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections - stack, heap, text and data.



Layout of a process

S.N.	Component & Description
1	Stack - The process Stack contains the temporary data such as method/function parameters, return address and local variables.
2	Heap - This is dynamically allocated memory to a process during its run time.
3	Text - This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4	Data - This section contains the global and static variables.

Program

A program is a piece of code which may be a single line or millions of lines.

A computer program is usually written by a computer programmer in a programming language.

A computer program is a collection of instructions that performs a specific task when executed by a computer.

Algorithm

A part of a computer program that performs a well-defined task is known as an algorithm.

Software

A collection of computer programs, libraries and related data are referred to as a software.

Process Life Cycle / State

When a process executes, it passes through different states.

These stages may differ in different operating systems, and the names of these states are also not standardized.

In general, a process can have one of the following five states at a time.

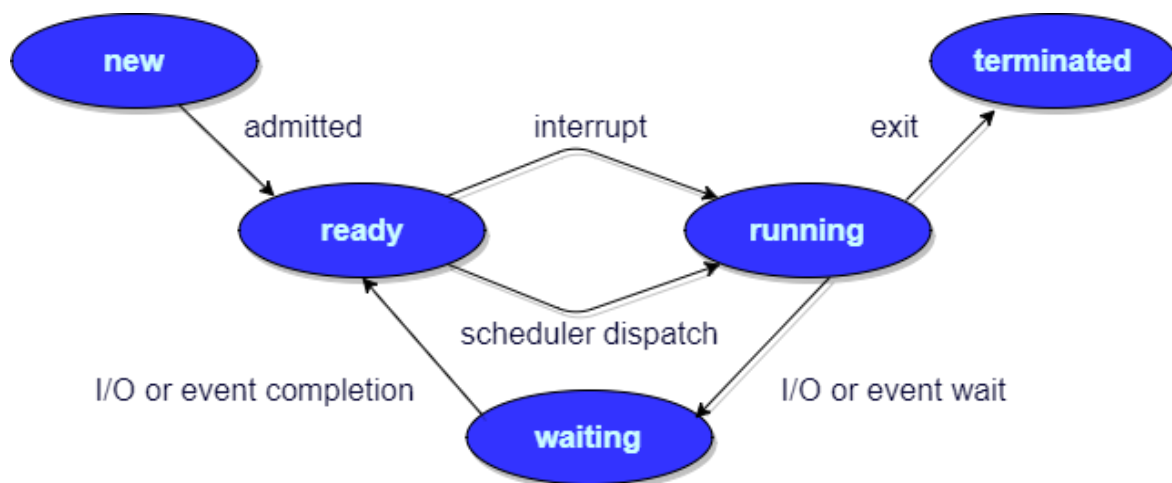
NEW - The process is being created.

READY - The process is waiting to be assigned to a processor.

RUNNING - Instructions are being executed.

WAITING - The process is waiting for some event to occur (such as an I/O completion or reception of a signal).

TERMINATED - The process has finished execution.



Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process.

A PCB keeps all the information needed to keep track of a process.

Process State - The current state of the process i.e., whether it is ready, running, waiting, or whatever.

Process ID - Unique identification for each of the process in the operating system.

Program Counter - Program Counter is a pointer to the address of the next instruction to be executed for this process.

CPU registers - Various CPU registers where process need to be stored for execution for running state.

Memory management information - This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.

Accounting information - This includes the amount of CPU used for process execution, time limits, execution ID etc.

IO status information - This includes a list of I/O devices allocated to the process.



The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates.

PROCESS SCHEDULING

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process.

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Scheduling Queues.

It refers to queues of process or devices.

Job queue - This queue keeps all the processes in the system.

Ready queue - This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.

Device queues - The processes which are blocked due to unavailability of an I/O device constitute this queue.

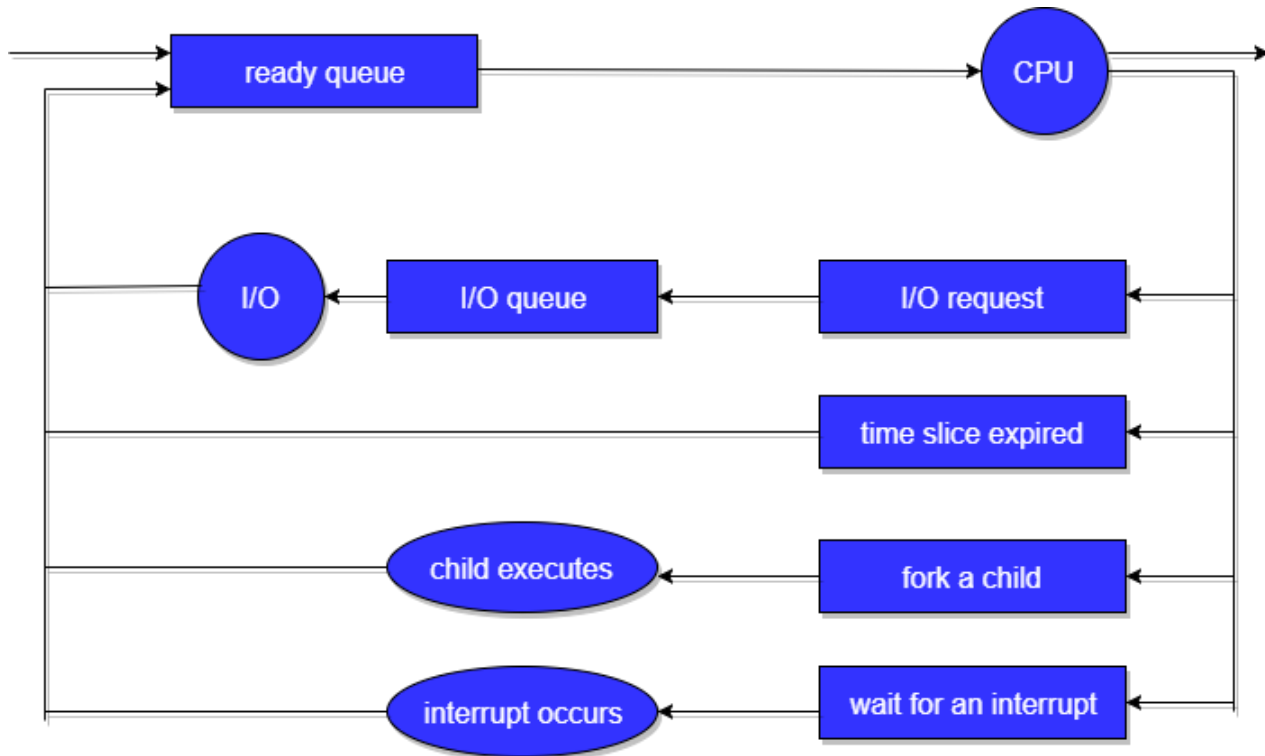
Queue is represented by **rectangular box**.

Circle represents the **resources** that serve the queues.

Arrows indicate the **process flow** in the system.

A new process is initially put in the Ready queue. It waits in the ready queue until it is selected for execution (or dispatched). Once the process is assigned to the CPU and is executing, one of the following several events can occur:

- The process could issue an I/O request, and then be placed in the I/O queue.
- The process could create a new sub process and wait for its termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.



In the first two cases, the process eventually switches from the waiting state to the ready state, and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

Two-State Process Model

Two-state process model refers to running and non-running states

Running - When a new process is created, it enters into the system as in the running state.

Not Running - Processes that are not running are kept in queue, waiting for their turn to execute.

Schedulers

Schedulers are special system software which handles process scheduling in various ways.

Its main task is to select the jobs to be submitted into the system and to decide which process to run.

Schedulers are of three types:

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

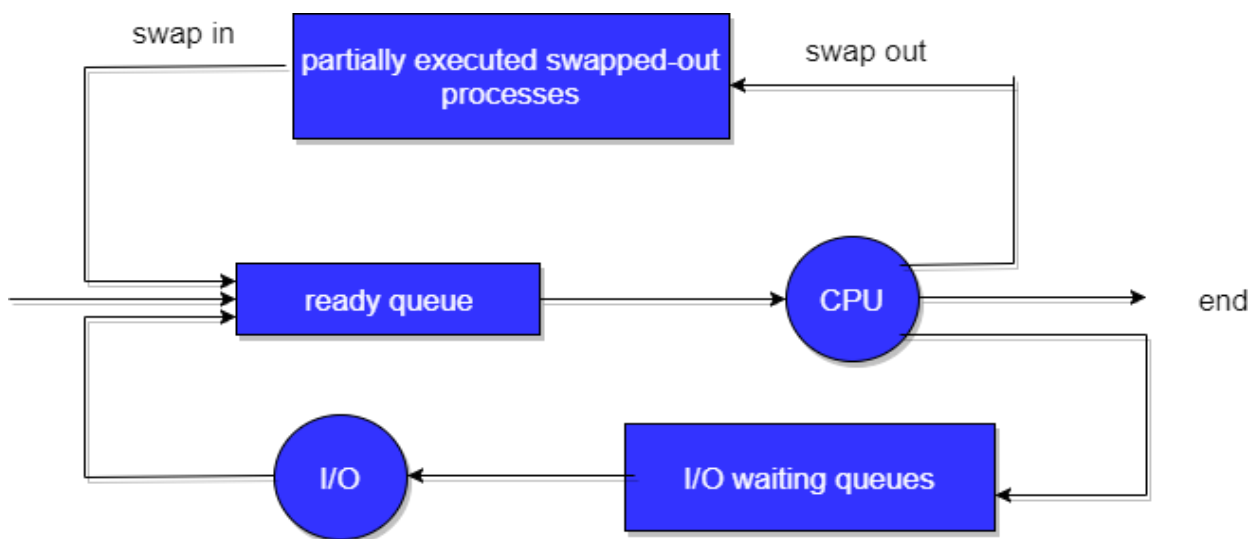
Long-Term Scheduler - It is also called a **job scheduler**. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming.

Short-Term Scheduler - It is also called as **CPU scheduler**. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them. Short-term schedulers, also known as **dispatchers**, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

Medium-Term Scheduler - Medium-term scheduling is a part of **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be **swapped out or rolled out**.



Comparison among Scheduler

S.N.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution.	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

OPERATIONS ON PROCESS

The two major operation provided by OS are Process Creation and Process Termination.

Process Creation

The system calls, fork is used by processes may create other processes.

The process which creates other process, is termed the parent process, while the created sub-process is termed its child.

The child process is a duplicate of the parent process.

Each process is given an integer identifier, termed as process identifier, or PID.

The parent PID (PPID) is also stored for each process.

Process Termination

A process terminates when it finishes executing its last statement.

Its resources are returned to the system.

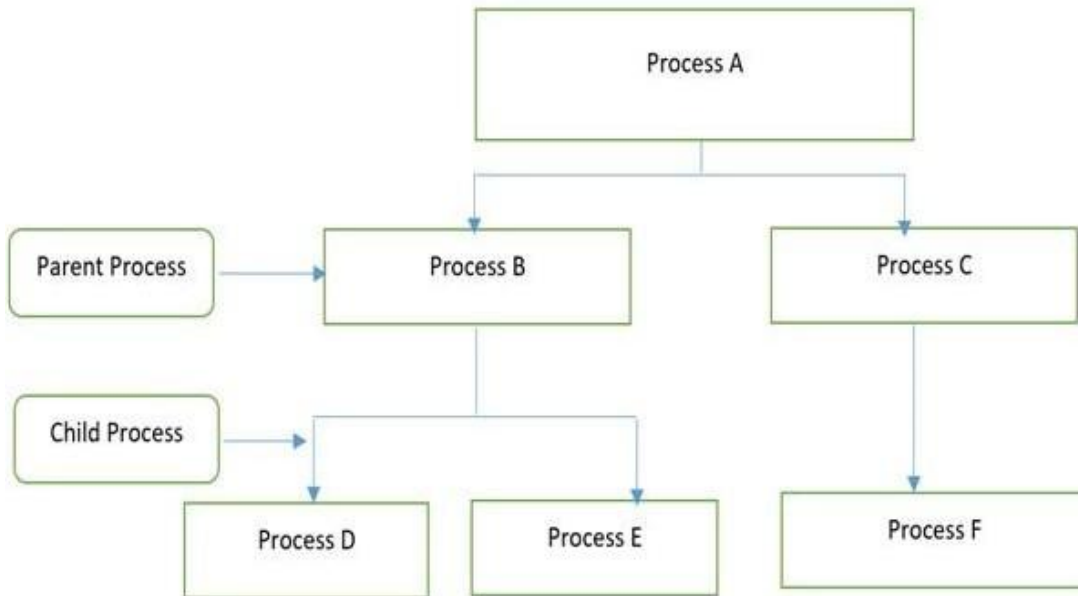
The new process terminates the existing process, usually due to following reasons:

Normal Exist - Most processes terminates because they have done their job. This call is exist in UNIX.

Error Exist - When process discovers a fatal error. For example, a user tries to compile a program that does not exist.

Fatal Error - An error caused by process due to a bug in program for example, executing an illegal instruction, referring non-existing memory or dividing by zero.

Killed by Another Process - A process executes a system call telling the Operating Systems to terminate some other process.



INTER-PROCESS COMMUNICATION

Inter Process Communication (IPC) refers to a mechanism, for processes to communicate and to synchronize their actions.

Interprocess communication (IPC) is a set of programming interfaces that allow a programmer to coordinate activities among different program processes that can run concurrently in an operating system.

Message system - processes communicate with each other without resorting to shared variables.

IPC facility provides two operations:

Send (message) – message size fixed or variable

Receive (message)

If P and Q wish to communicate, they need to:

Establish a **communication link** between them

Exchange **messages** via send/receive

Implementation of communication link

Physical (e.g., shared memory, hardware bus)

Logical (e.g., logical properties)

Message Passing System

Messages sent by a process can be of either fixed or variable size. Implementation is straight forward.

Several methods for logically implementing a link and the send /receive operations.

- Direct / Indirect Communication
- Symmetric / Asymmetric Communication
- Automatic / Explicit buffering
- Synchronous / Asynchronous Communication

Direct / Indirect Communication

Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication.

Direct communication

Process wants to communicate must name each other explicitly:

send (P, message) - send a message to process P

receive (Q, message) - receive a message from process Q

Indirect Communication

Messages are directed and received from mailboxes (also referred to as ports).

Each mailbox has a unique id.

Processes can communicate only if they share a mailbox.

send (A, message) - send a message to mailbox A

receive (A, message) - receive a message from mailbox A

Symmetric / Asymmetric Communication

Symmetric Communication

Both recipient and sender must name the other for communication.

send (P, message) - Send a message to process P.

receive (Q, message) - Receive a message from process Q.

Asymmetric Communication

The sender names the recipient; the recipient is not required to name the sender.

send (P, message) - Send a message to process P.

receive (id, message) - Receive a message from any process; the variable id is set to the name of the process with which communication has taken place.

Automatic / Explicit buffering

Automatic buffering

Provides a queue with indefinite length, thus ensuring the sender will never have to block while waiting to copy a message.

Explicit buffering

Specifies how large the buffer is. In this situation, the sender may be blocked while waiting for available space in the queue.

Synchronous / Asynchronous Communication

Synchronous Communication

P and Q have to wait for each other (one blocks until the other is ready).

Asynchronous Communication

Underlying system buffers the messages so P and Q don't have to wait for each other.

CPU SCHEDULING

CPU scheduling is a process which allows one process to use the CPU while the execution of another process is on hold (in waiting state) due to unavailability of any resource like I/O etc, thereby making full use of CPU. The aim of CPU scheduling is to make the system efficient, fast and fair.

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the short-term scheduler (or CPU scheduler). The scheduler selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them.

SCHEDULING CRITERIA

There are many different criteria's to check when considering the "best" scheduling algorithm, they are:

CPU Utilization

To make out the best use of CPU and not to waste any CPU cycle, CPU would be working most of the time. (Ideally 100% of the time).

Throughput

It is the total number of processes completed per unit time or rather say total amount of work done in a unit of time.

$$\text{Throughput} = \frac{\text{Number of processes completed}}{\text{Total amount of time required to complete processes}}$$

Turnaround Time

It is the amount of time taken to execute a particular process, i.e. the interval from time of submission of the process to the time of completion of the process.

Waiting Time

The sum of the periods spent waiting in the ready queue amount of time a process has been waiting in the ready queue to acquire get control on the CPU.

Response Time

Amount of time it takes from when a request was submitted until the first response is produced. Remember, it is the time till the first response and not the completion of process execution.

Two Scheduling Philosophies:

Preemptive Scheduling

In preemptive mode, currently running process may be interrupted and moved to the ready State by the operating system.

Non-preemptive Scheduling

In non-preemptive mode, once if a process enters into running state, it continues to execute until it terminates or blocks itself to wait for Input/Output or by requesting some operating system service.

SCHEDULING ALGORITHMS

To decide which process to execute first and which process to execute last to achieve maximum CPU utilisation, they are:

- First Come First Serve (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- Round Robin (RR) Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

First Come First Serve (FCFS) Scheduling

Jobs are executed on first come, first serve basis.

It is a non-preemptive, pre-emptive scheduling algorithm.

Easy to understand and implement.

The process which arrives first, gets executed first, or we can say that the process which requests the CPU first gets the CPU allocated first.

Its implementation is based on FIFO (First in First out) Queue data structure, where the data element which is added to the queue first, is the one who leaves the queue first.

This is used in Batch Systems.

Example Problem:

1. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	24
P2	3
P3	3

What is the average waiting time and average turnaround time for these processes?

Gantt chart

P1	P2	P3	
0	24	27	30

Average Waiting Time

Waiting Time

Waiting Time = Completion Time - Burst Time - Arrival Time

Process	Waiting Time
P1	$24 - 24 - 0 = 0$
P2	$27 - 3 - 0 = 24$
P3	$30 - 3 - 0 = 27$

Average Waiting Time = $0 + 24 + 27 / 3 = 17$ ms

Average Turn around Time

Turn around Time

Turn around Time = *Completion Time - Arrival Time*

Process	Turn around Time
P1	$24 - 0 = 24$
P2	$27 - 0 = 27$
P3	$30 - 0 = 30$

Average Turn around Time = $24 + 27 + 30 / 3 = 27$ ms

Throughput = $\frac{\text{Number of processes completed}}{\text{Total amount of time required to complete processes}}$

= $3 / 30 = 0.1$ processes per millisecond

Additional Problems:

2. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	14
P2	2
P3	4

What is the average waiting time and average turnaround time for these processes?

3. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	3
P2	6
P3	4
P4	2

What is the average waiting time and average turnaround time for these processes?

4. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	21
P2	3
P3	6
P4	2

What is the average waiting time and average turnaround time for these processes?

5. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	4
P2	7
P3	3
P4	3
P5	3

What is the average waiting time and average turnaround time for these processes?

6. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	5
P2	10
P3	8
P4	3

What is the average waiting time and average turnaround time for these processes?

Shortest-Job-First (SJF) / Shortest Job Next (SJN) Scheduling

Shortest Job First scheduling works on the process with the shortest burst time or duration first.

If two processes are available at the same time, FCFS is used to break the tie.

Best approach to minimize waiting time.

This is used in Batch Systems.

It is of two types:

- Non Pre-emptive
- Pre-emptive

Non Pre-emptive

Schedules the processes according to their burst time.

Example Problem:

1. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	5
P2	10
P3	8
P4	3

What is the average waiting time and average turnaround time for these processes?

Gantt chart

P4	P1	P3	P2	
0	3	8	16	26

Average Waiting Time

Waiting Time

Waiting Time = Completion Time – Burst Time – Arrival Time

Process	Waiting Time
P1	$8 - 5 - 0 = 3$
P2	$26 - 10 - 0 = 16$
P3	$16 - 8 - 0 = 8$
P4	$3 - 3 - 0 = 0$

Average Waiting Time = $3 + 16 + 8 + 0 / 4 = 6.7$ ms

Average Turn around Time

Turn around Time

Turn around Time = Completion Time - Arrival Time

Process	Turn around Time
P1	$8 - 0 = 8$
P2	$26 - 0 = 26$
P3	$16 - 0 = 16$
P4	$3 - 0 = 3$

Average Turn around Time = $8 + 26 + 16 + 3 / 4 = 13.25$ ms

Throughput = $\frac{\text{Number of processes completed}}{\text{Total amount of time required to complete processes}}$

$= 4 / 26 = 0.15$ processes per millisecond

Additional Problems:

2. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	21
P2	3
P3	6
P4	2

What is the average waiting time and average turnaround time for these processes?

3. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

What is the average waiting time and average turnaround time for these processes?

4. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	5
P2	2
P3	6
P4	4

What is the average waiting time and average turnaround time for these processes?

5. Consider the following set of processes that arrive at time 0, with the length of CPU burst given in milliseconds.

Process	Burst Time
P1	3
P2	6
P3	4
P4	2

What is the average waiting time and average turnaround time for these processes?

6. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Arrival Time
P1	7	0
P2	4	1
P3	1	4
P4	4	5

What is the average waiting time and average turnaround time for these processes?

Pre-emptive

Pre-emption occurs when a new process arrives in the ready queue that has a predicted burst time shorter than the time remaining in the process whose burst is currently on the CPU. Preemptive SJF is sometimes referred to as ***shortest remaining time first scheduling***. i.e. It is preemptive mode of SJF algorithm in which jobs are schedule according to shortest remaining time.

Example Problem:

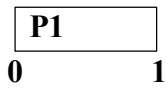
1. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Arrival Time
P1	8	0
P2	4	1
P3	9	2
P4	5	3

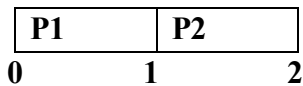
What is the average waiting time and average turnaround time for these processes?

Gantt chart

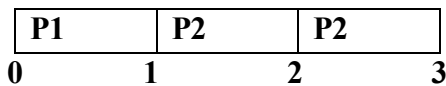
P1 starts at time 0 ms



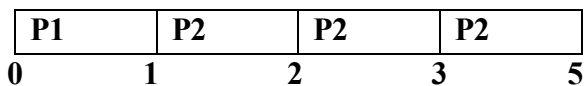
At 1 ms, P1 as 7 ms BT and compared with P2 as 4 ms BT, so we choose P2



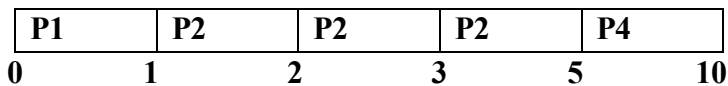
At 2 ms, P1 as 7 ms BT whereas P2 as 3 ms BT and P3 as 9 ms BT, so we choose P2



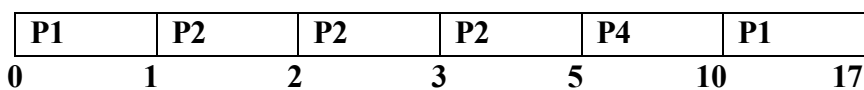
At 3 ms, P1 as 7 ms BT whereas P2 as 2 ms BT, P3 as 9 ms BT and P4 as 5 ms BT, so we choose P2



At 5 ms, P1 as 7 ms BT whereas P2 as 0 ms BT, P3 as 9 ms BT and P4 as 5 ms BT, so we choose P4



Now P2 and P4 are over, P1 as 7 ms BT, P3 as 9 ms BT, so we choose P1



Finally P1, P2 and P4 are over; P3 is remaining as 9 ms to be scheduled

P1	P2	P2	P2	P4	P1	P3	
0	1	2	3	5	10	17	26

Average Waiting Time

Waiting Time

Waiting Time = Completion Time – Burst Time – Arrival Time

Process	Waiting Time
P1	$17 - 8 - 0 = 9$
P2	$5 - 4 - 1 = 0$
P3	$26 - 9 - 2 = 15$
P4	$10 - 5 - 3 = 2$

Average Waiting Time = $9 + 0 + 15 + 2 / 4 = 6.5$ ms

Average Turn around Time

Turn around Time

Turn around Time = Completion Time - Arrival Time

Process	Turn around Time
P1	$17 - 0 = 17$
P2	$5 - 1 = 4$
P3	$26 - 2 = 24$
P4	$10 - 3 = 7$

Average Turn around Time = $17 + 4 + 24 + 7 / 4 = 13$ ms

Throughput =
$$\frac{\text{Number of processes completed}}{\text{Total amount of time required to complete processes}}$$

$= 4 / 26 = 0.15$ processes per millisecond

Additional Problems:

2. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Arrival Time
P1	6	0
P2	2	1
P3	8	2
P4	4	3

What is the average waiting time and average turnaround time for these processes?

3. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Arrival Time
P1	21	0
P2	3	1
P3	6	2
P4	2	3

What is the average waiting time and average turnaround time for these processes?

Priority Scheduling

Priority is assigned for each process.

Process with highest priority is executed first and so on.

Processes with same priority are executed in FCFS manner.

Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.

Example Problem:

1. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	21	2
P2	3	1
P3	6	4
P4	2	3

What is the average waiting time and average turnaround time for these processes?

Gantt chart

P2	P1	P4	P3	
0	3	24	26	32

Average Waiting Time

Waiting Time

Waiting Time = Completion Time – Burst Time – Arrival Time

Process	Waiting Time
P1	$24 - 21 - 0 = 3$
P2	$3 - 3 - 0 = 0$
P3	$32 - 6 - 0 = 26$
P4	$26 - 2 - 0 = 24$

Average Waiting Time = $3 + 0 + 26 + 24 / 4 = 13.25$ ms

Average Turn around Time

Turn around Time

Turn around Time = *Completion Time - Arrival Time*

Process	Turn around Time
P1	$24 - 0 = 24$
P2	$3 - 0 = 3$
P3	$32 - 0 = 32$
P4	$26 - 0 = 26$

Average Turn around Time = $24 + 3 + 32 + 26 / 4 = 21.25$ ms

Throughput = $\frac{\text{Number of processes completed}}{\text{Total amount of time required to complete processes}}$

= $4 / 32 = 0.125$ processes per millisecond

Additional Problems:

2. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

What is the average waiting time and average turnaround time for these processes?

3. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	5	3
P2	10	2
P3	8	4
P4	3	1

What is the average waiting time and average turnaround time for these processes?

4. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	10	2
P2	7	1
P3	20	3
P4	30	4

What is the average waiting time and average turnaround time for these processes?

5. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	3	2
P2	6	4
P3	4	1
P4	2	3

What is the average waiting time and average turnaround time for these processes?

6. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds.

Process	Burst Time	Priority
P1	5	3
P2	2	4
P3	6	1
P4	4	2

What is the average waiting time and average turnaround time for these processes?

Round Robin Scheduling

Round Robin is the preemptive process scheduling algorithm.

Each process is provided a fix time to execute, it is called a **quantum**.

Once a process is executed for a given time period, it is pre-empted and other process executes for a given time period.

Every process gets executed in a cyclic way.

Context switching is used to save states of pre-empted processes.

Example Problem:

1. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds. The time quantum given is 2 ms.

Process	Burst Time
P1	5
P2	2
P3	6
P4	4

What is the average waiting time and average turnaround time for these processes?

Gantt chart

P1	P2	P3	P4	P1	P3	P4	P1	P3	
0	2	4	6	8	10	12	14	15	17

Average Waiting Time

Waiting Time

$$\text{Waiting Time} = \text{Completion Time} - \text{Burst Time} - \text{Arrival Time}$$

Process	Waiting Time
P1	$15 - 5 - 0 = 10$
P2	$4 - 2 - 0 = 2$
P3	$17 - 6 - 0 = 11$
P4	$14 - 4 - 0 = 10$

Average Waiting Time = $10 + 2 + 11 + 10 / 4 = 8.25$ ms

Average Turn around Time

Turn around Time

Turn around Time = *Completion Time - Arrival Time*

Process	Turn around Time
P1	$15 - 0 = 15$
P2	$4 - 0 = 4$
P3	$17 - 0 = 17$
P4	$14 - 0 = 14$

Average Turn around Time = $15 + 4 + 17 + 14 / 4 = 12.5$ ms

$$\begin{aligned}
 \text{Throughput} &= \frac{\text{Number of processes completed}}{\text{Total amount of time required to complete processes}} \\
 &= 4 / 17 = 0.235 \text{ processes per millisecond}
 \end{aligned}$$

Additional Problems:

2. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds. The time quantum given is 2 ms.

Process	Burst Time
P1	3
P2	6
P3	4
P4	2

What is the average waiting time and average turnaround time for these processes?

3. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds. The time quantum given is 4 ms.

Process	Burst Time
P1	24
P2	3
P3	3

What is the average waiting time and average turnaround time for these processes?

4. Consider the following set of processes in the same order of arrival, with the length of CPU burst given in milliseconds. The time quantum given is 5 ms.

Process	Burst Time
P1	25
P2	5
P3	5

What is the average waiting time and average turnaround time for these processes?

Multilevel Queue Scheduling

A multilevel queue scheduling algorithm partitions the ready queue in several separate queues.

In a multilevel queue scheduling processes are permanently assigned to one queues.

The processes are permanently assigned to one another, based on some property of the process, such as

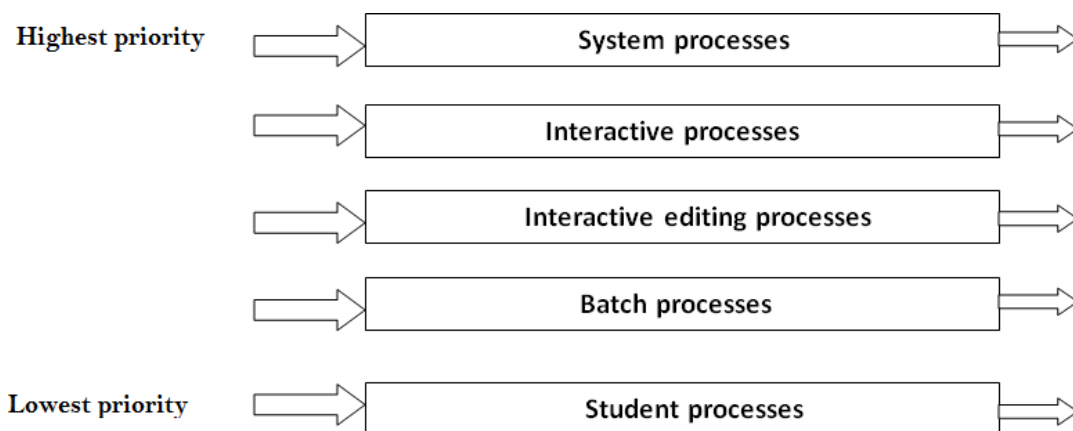
- Memory size
- Process priority
- Process type

Algorithm choose the process from the occupied queue that has the highest priority, and run that process either

- Preemptive or
- Non-preemptively

Each queue has its own scheduling algorithm or policy.

The foreground queue can be scheduled by using a round-robin algorithm (80% of the CPU time) while the background queue is scheduled by a first come first serve algorithm (20% of the CPU time).



Let us take an example of a multilevel queue scheduling algorithm with five queues:

1. System process
2. Interactive processes
3. Interactive editing processes
4. Batch processes
5. Student processes

All the five processes have their own queue.

System process: there is various kind of process in the operating system. The operating system itself has own process to run called system process

Interactive process: the interactive process is online game or listing music (there should be the same kind of interaction)

Interactive editing process: the interactive editing process is Data or Image editing (there should be the same kind of interaction)

Batch system: we submit the job to the processor and take the result later.

Student process: normal student processes (Application) comes inside student process.

The system process will get the highest priority and the student process gets the lowest priority.

There are many types of the process we can't put them in one queue and get the result.

This problem is resolved by the multilevel queue scheduling.

Disadvantage:

In multilevel queue scheduling, the main disadvantage is the Starvation problem for lowest level process

Starvation:

Lower priority process never executes. or wait for the long amount of time because of lower priority or highest priority process taking a large amount of time.

Advantage:

We can apply separate scheduling for various kind of process like

System process - FCFS scheduling algorithm

Interactive process - SJF scheduling

Batch process - Round Robin

Student Process - Priority scheduling

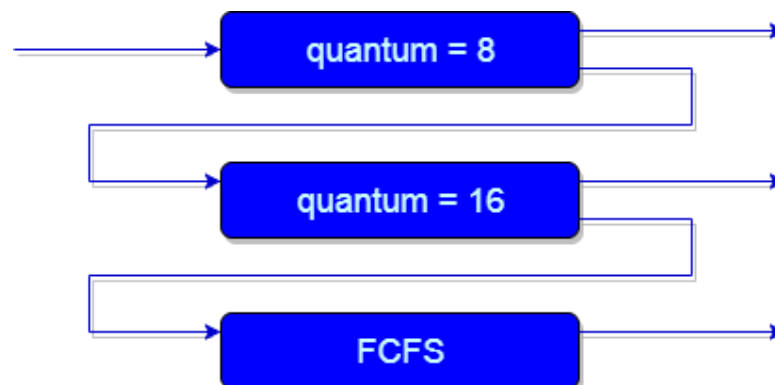
Multilevel Feedback Queue Scheduling

Generally, we see in a ***multilevel queue scheduling algorithm*** processes are permanently stored in one queue in the system and do not move between the queue. There is some separate queue for foreground or background processes but the processes do not move from one queue to another queue and these processes do not change their foreground or background nature, these type of arrangement has the advantage of low scheduling but it is inflexible in nature.

Multilevel feedback queue scheduling it allows a process to move between the queues. This process is separate with different CPU burst time. If a process uses too much CPU time then it will be moved to the lowest priority queue. This idea leaves I/O bound and interactive processes in the higher priority queue. Similarly, the process which waits too long in a lower priority queue may be moved to a higher priority queue. This form of aging prevents starvation.

The multilevel feedback queue scheduler has the following parameters:

1. The number of queues
2. The scheduling algorithm for each queue
3. The method used to demote/downgrade processes to lower priority queues
4. The method used to promote/upgrade processes to higher priority queues
5. The method used to determine which queue a process will enter



Let's consider there are three queues Q1, Q2 and Q3.

- **Q1** Time quantum 8 milliseconds
- **Q2** Time quantum 16 milliseconds
- **Q3** FCFS

Criteria of Multilevel Feedback Queue:

- The scheduler first executes all processes in Q1.
- Only when Q1 is empty will it execute processes in Q2.
- Similarly, processes in Q3 will be executed only if Q1 and Q2 are empty.
- A process that arrives for Q2 will pre-empt a process in Q3.
- A process that arrives for Q3 will, in turn, pre-empt a process in Q3.

Explanation

Any process entering the ready queue is put in Q1. Then, A process in Q1 is given a time quantum of 8 milliseconds. If it does not finish within time, it is moved to the tail of Q2.

If Q1 is empty, the process at the head of Q2 is given a quantum of 16 milliseconds. Or, if it does not complete, it is pre-empted and is put into Q3.

Processes in Q3 are run on an FCFS basis, only when Q1 and Q2 are empty.

Aging promotes lower priority process to the next higher priority queue after a suitable interval of time.

Advantages

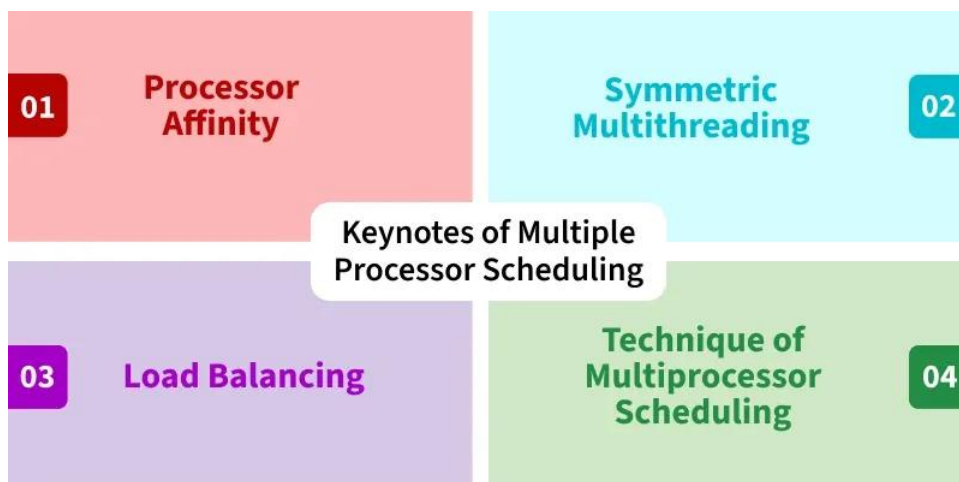
A process that waits too long in a lower priority queue may be moved to a higher priority queue.

Disadvantages

Moving the process around queue produces more CPU overhead.

Multiple Processor Scheduling

CPU Scheduling is a mechanism by which an operating system decides which task or process should execute on the CPU at any given moment. When a system contains more than one CPU, Multiple-Processor Scheduling involves distributing tasks across multiple processors. This enables several tasks to be processed in parallel, improving performance significantly.



Key Challenges:

- Deciding which CPU handles which task.
- Balancing workloads between processors to avoid idle CPUs or overloaded processors.

Note: In multiple processor scheduling, there are cases when the processors are identical i.e. Homogenous, in terms of their functionality, we can use any processor available to run any process in the queue.

Approaches to Multiple-Processor Scheduling

Asymmetric Multiprocessing (AMP)

- One processor acts as a Master Server handling scheduling decisions and I/O operations.
- Other processors execute only user code.
- Simple design, reduces data-sharing complexity.

Symmetric Multiprocessing (SMP)

- Each processor can schedule tasks independently.
- Two models of task queues:

Common Ready Queue: All CPUs access a shared queue of ready processes.

Private Ready Queues: Each CPU has its own queue of ready processes.

Important Concepts in Multiprocessor Scheduling

1. Processor Affinity

- A process prefers to run on the same processor it previously ran on.
- **Benefits:** Data stays in the CPU cache & reduces cache misses and improves performance.

Types of Affinity:

- **Soft Affinity:** OS attempts (but does not guarantee) to keep a process on the same CPU.
- **Hard Affinity:** Restricts the process to run only on a specific subset of processors using system calls (e.g., `sched_setaffinity()` in Linux).

2. Load Balancing

Essential to avoid:

- Overloaded processors with long queues.
- Idle processors while other processors are busy.

Two approaches:

1. **Push Migration:** A processor actively moves tasks from itself to less busy processors to balance load.
2. **Pull Migration:** An idle processor pulls tasks from busy processors to balance its workload.

3. Multicore Processors

- Multiple processor cores on a single physical chip.
- Each core appears as an independent processor to the OS.

Challenge: Memory Stall

- Happens when a processor waits for data not present in cache.
- Up to 50% of time can be wasted waiting for memory.

Solution: Multithreading within Cores

- **Coarse-Grained Multithreading:** Switch thread when long latency events occur (e.g., cache miss).
- **Fine-Grained Multithreading:** Switch threads every instruction cycle, keeping the pipeline filled with minimal overhead.

4. Virtualization and Threading

- Single physical CPU appears as multiple virtual CPUs to virtual machines.
- Host OS manages multiple guest OS instances.
- **Problem:** Time-slice distortion - guest OS time slices may not match real CPU cycles due to virtualization overhead.

Impacts: Poor response times for virtualized systems & Time-of-day clocks may be inaccurate.

Pros of Multiple-Processor Scheduling

- High throughput by parallel execution of tasks.
- Better resource utilization.
- Scalability for large applications and real-time systems.

Cons of Multiple-Processor Scheduling

- Complex implementation and maintenance.
- Difficult load balancing.
- Increased overhead due to process migration and affinity management.
- Virtualization may affect real-time guarantees.

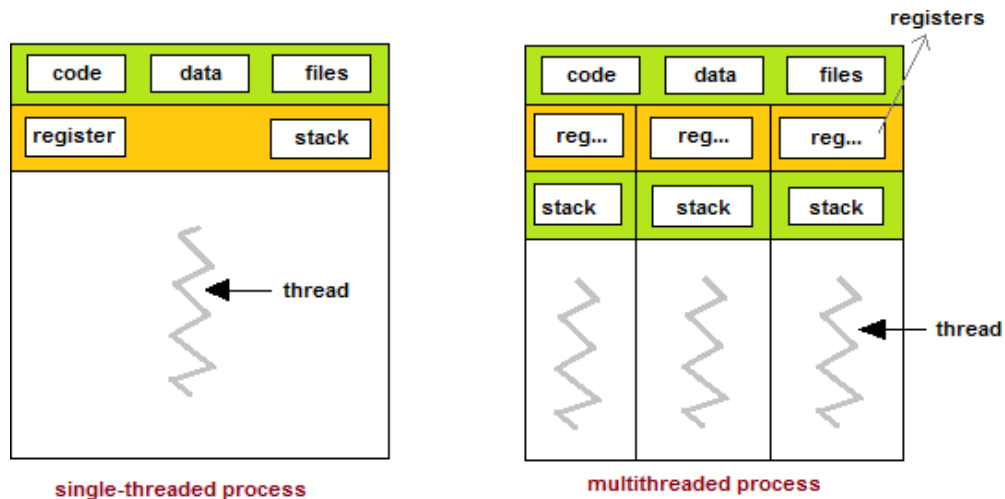
THREADS – OVERVIEW

A Thread is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID).

Traditional (heavyweight) processes have a single thread of control - There is one program counter, and one sequence of instructions that can be carried out at any given time.

A process can contain multiple threads.

A thread is also known as lightweight process.



Difference between Process and Thread

S.N.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

Advantages of Thread

- Threads minimize the context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- It is more economical to create and context switch threads.
- Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

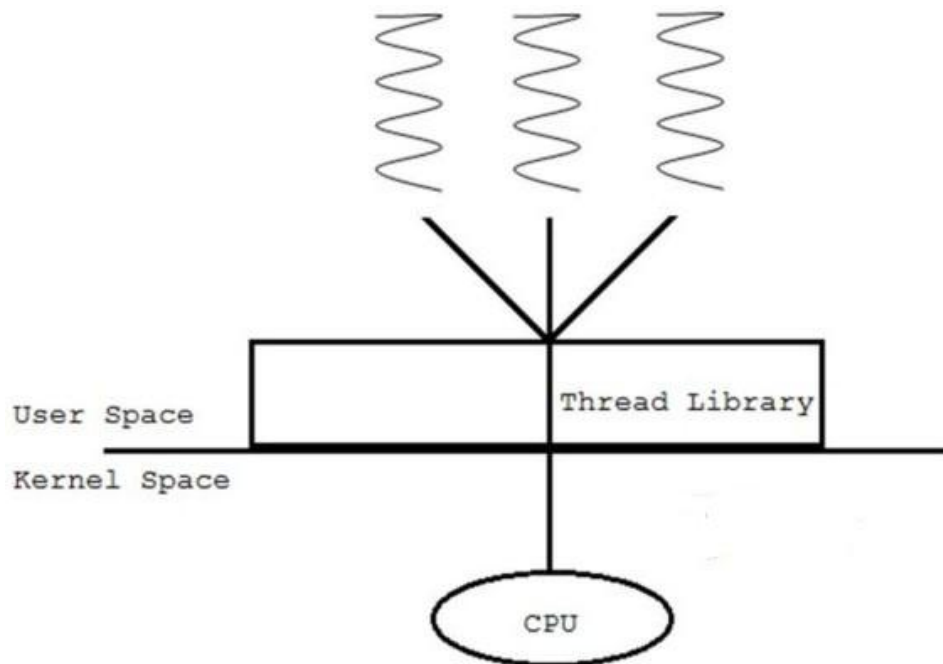
Types of Thread

Threads are implemented in following two ways:

- **User Level Threads** – User managed threads.
- **Kernel Level Threads** – Operating System managed threads acting on kernel, an operating system core.

User threads are above the kernel and without kernel support. These are the threads that application programmers use in their programs.

Kernel threads are supported within the kernel of the OS itself. All modern OSs support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.



Thread Libraries

Thread libraries provide programmers with API for creation and management of threads.

Thread libraries may be implemented either in user space or in kernel space.

The user space involves API functions implemented solely within the user space, with no kernel support.

The kernel space involves system calls, and requires a kernel with thread library support.

Implementations of Thread

POSIX Pthreads may be provided as either a user or kernel library, as an extension to the POSIX standard.

Win32 threads are provided as a kernel-level library on Windows systems.

Java threads Since Java generally runs on a Java Virtual Machine, the implementation of threads is based upon whatever OS and hardware the JVM is running on, i.e. either Pthreads or Win32 threads depending on the system.

Linux threads Linux does not distinguish between processes and threads - It uses the more generic term "tasks". The traditional fork () system call completely duplicates a process (task), as described earlier. An alternative system call, clone () allows for varying degrees of sharing between the parent and child tasks.

Benefits of Multithreading

Responsiveness, one thread may provide rapid response while other threads are blocked or slowed down doing intensive calculations.

Resource sharing, hence allowing better utilization of resources.

Economy, Creating and managing threads become easier.

Scalability, One thread runs on one CPU. In Multithreaded processes, threads can be distributed over a series of processors to scale.

Context Switching is smooth. Context switching refers to the procedure followed by CPU to change from one task to another.

MULTITHREADING MODELS

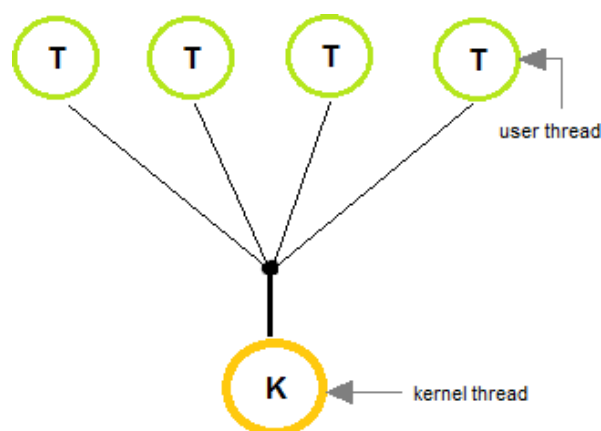
The user threads must be mapped to kernel threads, by one of the following strategies:

- Many to One Model
- One to One Model
- Many to Many Model

Many to One Model

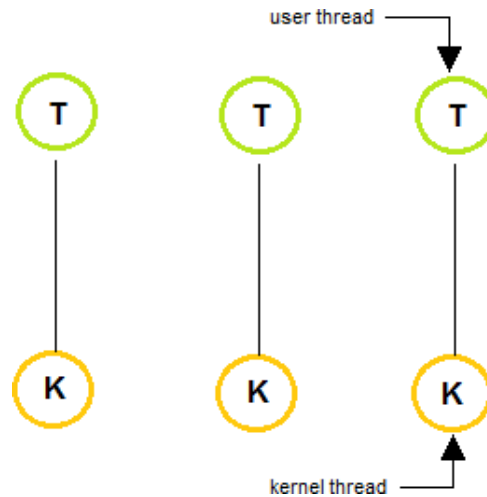
In the many to one model, many user-level threads are all mapped onto a single kernel thread.

Thread management is handled by the thread library in user space, which is efficient in nature.



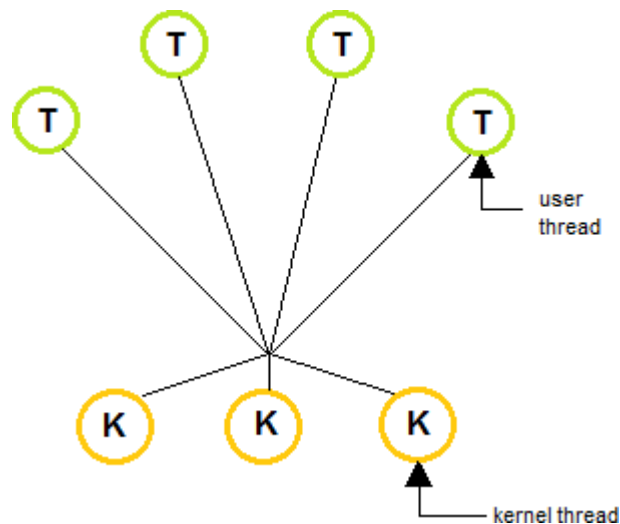
One to One Model

The one to one model creates a separate kernel thread to handle each and every user thread. Most implementations of this model place a limit on how many threads can be created. Linux and Windows from 95 to XP implement the one-to-one model for threads.

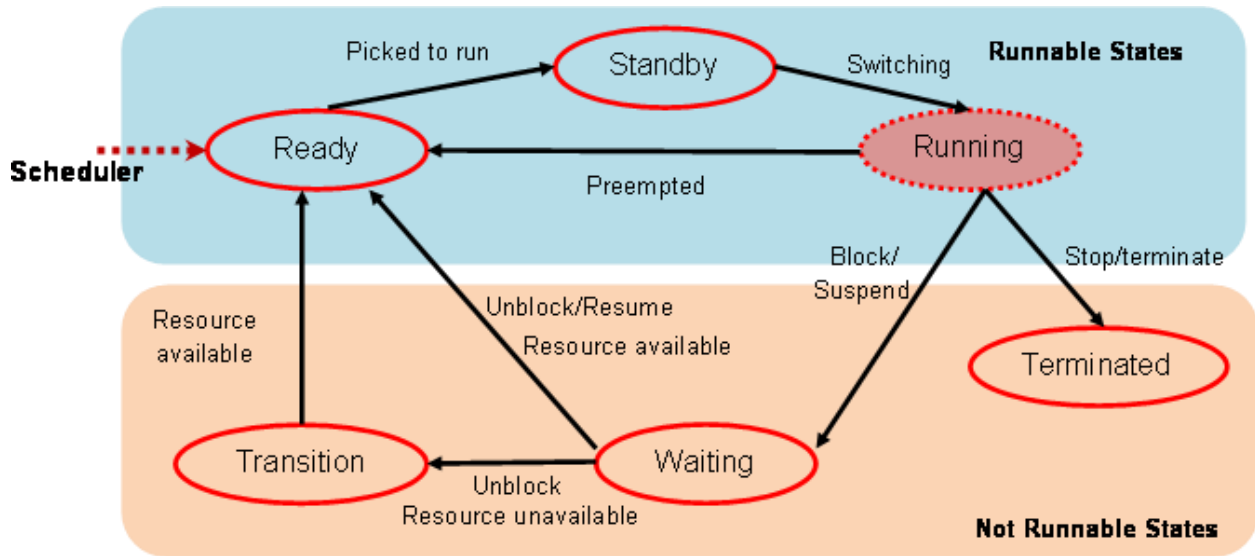


Many to Many Model

The many to many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models. Users can create any number of the threads. Blocking the kernel system calls does not block the entire process. Processes can be split across multiple processors.



Thread States



Windows Thread States

A thread can be in the following states:

- **Ready** - it is prepared to run on the next available processor.
- **Running** - it is executing.
- **Standby** - it is about to run; only one thread may be in this state at a time.
- **Terminated** - it is finished executing.
- **Waiting** - it is not ready for the processor, when ready, it will be rescheduled.
- **Transition** - the thread is waiting for resources other than the processor,

THREADING ISSUES

The fork() and exec() System Calls

Q: If one thread forks, is the entire process copied, or is the new process single-threaded?

A: System dependant.

A: If the new process execs right away, there is no need to copy all the other threads. If it doesn't, then the entire process should be copied.

A: Many versions of UNIX provide multiple versions of the fork call for this purpose.

Signal Handling

Q: When a multi-threaded process receives a signal, to what thread should that signal be delivered?

A: There are four major options:

1. Deliver the signal to the thread to which the signal applies.
2. Deliver the signal to every thread in the process.
3. Deliver the signal to certain threads in the process.
4. Assign a specific thread to receive all signals in a process.

The best choice may depend on which specific signal is involved.

UNIX allows individual threads to indicate which signals they are accepting and which they are ignoring. However the signal can only be delivered to one thread, which is generally the first thread that is accepting that particular signal.

UNIX provides two separate system calls, *kill (pid, signal)* and *pthread_kill (tid, signal)*, for delivering signals to processes or specific threads respectively.

Windows does not support signals, but they can be emulated using Asynchronous Procedure Calls (APCs). APCs are delivered to specific threads, not processes.

Thread Cancellation

Threads that are no longer needed may be cancelled by another thread in one of two ways:

Asynchronous Cancellation cancels the thread immediately.

Deferred Cancellation sets a flag indicating the thread should cancel itself when it is convenient. It is then up to the cancelled thread to check this flag periodically and exit nicely when it sees the flag set.

(Shared) resource allocation and inter-thread data transfers can be problematic with asynchronous cancellation.

Thread-Local Storage (Thread-Specific Data)

Most data is shared among threads, and this is one of the major benefits of using threads in the first place.

However sometimes threads need thread-specific data also.

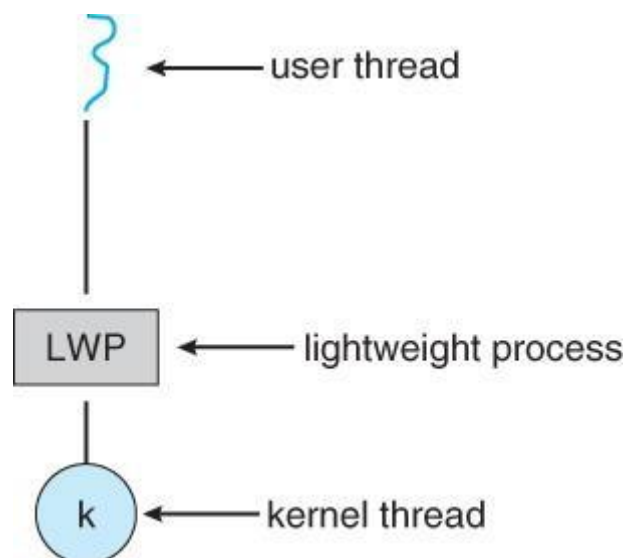
Most major thread libraries (pThreads, Win32, Java) provide support for thread-specific data, known as thread-local storage or TLS. Note that this is more like static data than local variables, because it does not cease to exist when the function ends.

Scheduler Activations

Many implementations of threads provide a virtual processor as an interface between the user thread and the kernel thread, particularly for the many-to-many or two-tier models.

This virtual processor is known as a "Lightweight Process", LWP.

There is a one-to-one correspondence between LWPs and kernel threads.



The number of kernel threads available, (and hence the number of LWPs) may change dynamically.

The application (user level thread library) maps user threads onto available LWPs.

Kernel threads are scheduled onto the real processor(s) by the OS.

The kernel communicates to the user-level thread library when certain events occur (such as a thread about to block) via an upcall, which is handled in the thread library by an ***upcall handler***.

The upcall also provides a new LWP for the upcall handler to run on, which it can then use to reschedule the user thread that is about to become blocked. The OS will also issue upcalls when a thread becomes unblocked, so the thread library can make appropriate adjustments.

If the kernel thread blocks, then the LWP blocks, which blocks the user thread.